Page No.:
Date: / /

Name: Chowdhya Anikumar A
SRN: PES1PG25CA044
Coursecode: UQ25CA653A

Q-1 Student result table

Functional dependencies:

SRN -> Studentname
subjectcode -> subjectname, facultyuniq
facultyuniq -> faculty
(SRN, subjectcode) -> marks

Candidate key:
(SRN, subjectcode)

Highest normal form:
1NF

Final Tables:
STUDENT (SRN, studentname)
SUBJECT (subjectcode, subjectname, facultyuniq)
faculty (facultyuniq, Faculty)
RESULT (SRN, subjectcode, marks)

Q-2 Orders Details - 3NF

CUSTOMER(Orderid, customername, customercity)
PRODUCT(Product, Price)
ORDERS(Orderid, Product)

Disadvantage of over-normalization:
increased joins reduce performance

Teacher's Signature:.....

Page No.:
Date : \ \

Q3 course - faculty table

candidate key:
course code

BCNF Decomposition
COURSE (courseCode, facultyId)
FACULTY (facultyId, facultyname)

BCNF violation Reason:
facultyId -> facultyname

Q4 Project assignment

current normal form
2NF

3NF tables:
PROJECT (projectId, projectname)
Assignment (empId, projectId, hoursworked)

Q5 Relation R(a,b,c,d)

candidate key -> a
transitive dependency -> a->b->c
Highest normal form: 2NF

Teacher's Signature:.....

Page No.:
Date : / /

Q-5 Library Books

1NF violation:
authors is multivalued

3NF tables:
BOOK (bookId, title, publisher)
BOOK-AUTHOR (bookId, author)
PUBLISHER (publisher, publisherAddress)

→ mongoDB

Q-7 E-commerce
customers
orders

{
"_id" : ObjectId(),
"customerId" : "C105",
"name" : "Arjun",
"email" : "arjun@gmail.com"
}

{
"_id" : ObjectId(),
"orderId" : "O-2005",
"customerId" : "C105",
"OrderDate" : ISODate(),

Teacher's Signature:.....

```
"order_items": [
  {
    "ProductId": "P1",
    "quantity": 2,
    "Price": 500.
  },
  {
    "ProductId": "P2",
    "quantity": 1,
    "Price": 1200
  }
]
}
```

0-2 Movie database

Design decision

embed reviews inside movie document

```
{
  "_id": ObjectId(),
  "movieId": "m101",
  "title": "Interstellar",
  "year": 2014,
  "reviews": [
    {
      "user": "amit",
      "rating": 9,
      "comment": "Excellent"
    },
    {
      "user": "suru",
      "rating": 8,
      "comment": "very good"
    }
  ]
}
```

Teacher's Signature:.....

```
db.movies.insertOne({
  movieId: "m101",
  title: "Interstellar",
  year: 2014,
  reviews: [
    { user: "Amit", rating: 9, comment: "Excellent" },
    { user: "Guru", rating: 8, comment: "very good" }
  ]
})
```

Q-3) Internship management system

student -> application (referencing)
application -> internship (referencing)
internship -> company (embedding)

-> db.student.createIndex({ srn: 1 })

-> db.application.createIndex({ studentSRN: 1, internshipId: 1 })

Q-4) index on email

db.users.createIndex({ email: 1 })

db.users.getIndexes()

Page No.:
Date : / /

Q5/ unique index on SRN

db.students.createIndex({SRN:1}, {unique:true})

Q6/ compound index on internship

-> db.internships.createIndex({company:1, status:1})

Q7/ Explain before and after indexing

-> before index

db.application.find({studentsrn: "SRN01"}).
explain("executionstats")

- collscan

->after index

db.application.createIndex({studentsrn:1})
db.application.find({studentsrn: "SRN01"}).
explain("executionstats")

- ixscan

Teacher's Signature:.....